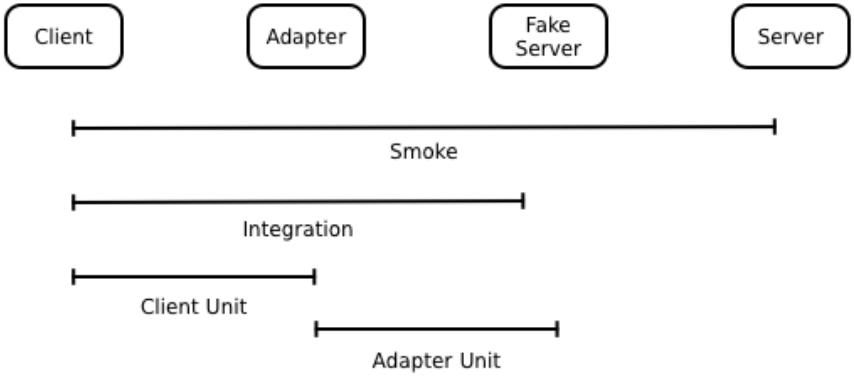
data to the API to be used by somebody else. In either case, you're dealing with a request and a response, even if the response is just a status code.

- The *server*, which for our purposes here is outside your application and reachable via some kind of network request (though many of the strategies in this chapter also apply even if the service isn't separated by the network).

I also introduce two characters to the story for design and testing purposes:

- A *fake server*, which intercepts HTTP requests during a test and returns a canned response object. You'll be using the VCR gem to manage your fake server (more about VCR shortly).
- An *adapter*, which is an object that sits between the client and the server to mediate access between them.

The diagram that follows shows the relationship between these objects and the tests you'll write using them.



There are several test types for you to use, each of which uses different combinations of these objects:

- A *smoke test*, which goes from the client all the way to the real server. In other words, it's a full end-to-end test of the entire interaction. You don't want to do this often, because it's slow, but it's useful to be able to guard against changes in the server API.
- An *integration test*, which goes from the client to the fake server. This tests the entire end-to-end functionality of your application but uses a stubbed response from the server. This will be our go-to strategy for integration-testing the external server.